GIL - Get in Line

Week 7 | From 3/17/26 – 3/23/26

Leader: Kyle Albuquerque

Disha Shetty ~20 hours **Time Spent**	**Tasks Completed** *Testing & Stability* ○ Tested and debugged Phase 4 frontend; resolved bugs before migration ○ Pushed updated frontend and backend to repositories with a copy repo protecting the main codebase *Supabase Migration* ○ Attempted initial migration via Codex; encountered critical errors and rolled back to last stable version ○ Restarted from stable baseline; migrated backend data layer from in-memory storage to Supabase PostgreSQL ○ Replaced all in-memory Maps with Supabase JS client queries across auth, events, and waitlist routes ○ Rewrote session management to use stateless HMAC-signed tokens; removed all demo data seeding ○ Verified table names, resolved case mismatches, and cleaned up duplicate tables from failed migration attempt (ongoing) ○ Wired party, account and event table info to persist to Supabase in real time *Auth & Frontend* ○ Updated auth middleware to verify tokens and resolve users from Supabase on every request ○ Replaced localStorage-based auth with live API calls; updated App.tsx handlers to async ○ Fixed frontend to load events from Supabase on login *AI Integration* ○ Integrated AI-powered wait time engine (Python/FastAPI) as a separate service on port 8001, still testing locally

	Deployment ○ Deployed Node.js backend to Render and frontend to Vercel ○ Configured environment variables across both platforms ○ Confirmed end-to-end signup flow works on deployed version **Challenges Faced** ○ Codex migration produced critical errors requiring a full rollback and restart from stable baseline ○ Multiple silent failures with no useful output: missing SESSION_SECRET, unresolved Supabase package, undefined config values, each required targeted debugging to identify ○ Frontend auth was entirely localStorage-based, meaning no actions were reaching the backend until fully rewritten ○ Table name case mismatches caused every database operation to silently fail ○ Each fix uncovered a new issue in the chain, making progress difficult to estimate throughout **Planned Tasks** ○ Migrate AI integration off localStorage to resolve sync errors with Supabase ○ Further integrate frontend and database components with improved reliability and error handling ○ Fix remaining Supabase sync errors and event display bug on login ○ Validate remaining flows end-to-end

Angela Lopez
 	
Time Spent
12 hours
	Tasks completed:
	• Weekly Meeting
	• Running tests from user side
	  ○ Joining capacity-based waitlists
	  ○ Joining table-based waitlists
	• Running tests from staff side
	  ○ Creating/editing/deleting events
	  ▪ Table-based
	  ▪ Capacity-based
	  ▪ Attendance-based
	• Tested private/public event creation from user and owner side

	• Worked on maintaining consistent UI throughout all pages
	  ○ Buttons, hamburger menus, etc.
	• Added public/private toggle for staff use when creating an event
	  ○ Added badges on the event card for easy view of which type of event it is
	• Fixed queue flow problem, correctly syncs with users who add themselves to a queue line
	• Changed event card layout for easier readability
	• Added date/time to attendance-type event
	• Changed default wait time for all events to 0 since attendance-type does not use
	• Edited “Your Events” page to only have one “+ Create Events” button when staff hasn’t created any events yet
	• Deleted “Back to Events” button in “Archived Events” page for better view/flow from page to page
	• Reformatted “Archived Events” event cards for better readability
	  ○ Deleted unnecessary information, changed layout, etc.
	
	Planned tasks for next week:
	• Create a way for users to access private events
	  ○ Generate specific 6-digit code associated with each event
	• Make sure all buttons and UI are consistent throughout app
	• Queue flow fixed and is correctly adding users to the queue lines but also adding them to the table waitlist list
	  ○ Make sure that capacity-type events not connected to table-type
	• Make single queue view the same as the multiple queue view on the user side
	• Filter reservations from earliest to latest in table-type view
	• Make sure attendance-type events are always private, user should not be able to access
	• When an event is edited and changed from private → public or vice versa, make sure it updates in the user view too
	
	Any issues or challenges:
	• As more features are developed, it takes the Figma AI longer to make changes or add other features
	• Signing in and out of user and staff accounts every time I am testing if both sides sync correctly, delays productivity
	• AI credits through Figma Make ran out so there were a few bugs I was unable to fix
	  ○ I was still able to make manual changes to the code and fix other bugs but there’s a few left over that require the AI to make sure it all syncs correctly

Neel Vavilapalli Time Spent :	Tasks completed:

12 hours	**Weekly Meeting:** Participated in the weekly team meeting to align on project milestones, coordinate backend integration with front-end requirements, and review the current development roadmap. Model optimization and features - Refined the predictive architecture and integrated several high-fidelity features to maximize accuracy: - **Dynamic Service Time Adjustments:** Implemented a rolling average calculation where the service time is updated every hour to reflect current operational speed. - **Adaptive No-Show Analytics:** Integrated a dynamic no-show rate that adjusts based on real-time trends and historical data. - **User Behavioral Weighting:** Developed a weighting system that adjusts individual user "reliability scores" every minute based on their specific no-show history. - **Grace Period Logic:** Integrated a buffer system to allow for arrival flexibility without penalizing the overall system flow. - **High-Frequency Recalculation:** Optimized the backend to trigger a full recalculation of wait times and user weights every 60 seconds, ensuring the most current data is always available. Core system and infrastructure - **Wait Time Prediction Model:** Finalized the core logic and successfully completed local implementation and unit testing. - **No-Show Prediction Model:** Completed the secondary predictive model designed to forecast attendance probabilities. - **Backend Environment Setup:** Configured the local development environment, including establishing secure **Supabase** connections for database management and real-time data streaming in preparation for db use in models. Planned tasks for next week: - **Database Integration:** Transition models from local environments to the live **Supabase** backend to begin handling persistent data.

	- **Deployment Support:** Assist in the orchestration of the deployment pipeline and address any environment-specific bottlenecks. - **Quality Assurance & Testing:** Lead end-to-end system testing to validate heuristic scoring and prediction accuracy in a live environment. Any issues or challenges:

Kyle Albuquerque
 	
Time Spent 14 hours
	Tasks completed:
	• Weekly Meeting
	• Researched and evaluated different deployment methods to determine the most efficient, scalable, and maintainable approach for hosting both frontend and backend services
	• Created, refactored, and optimized multiple backend functions to improve communication between the frontend and database, ensuring reliable data flow and cleaner architecture
	  ○ Implemented unique alphanumeric event IDs generated at event creation, allowing users to securely and easily join private events
	  ○ Fully automated multi-queue event generation, ensuring queues are created with correct parameters and minimal manual setup
	  ○ Enabled functionality for users to be seated and unseated at EventTables, improving real-time event management
	  ▪ Added timestamp tracking for when users are seated and unseated at tables, providing more accurate data for analytics and future AI modeling
	  ○ Implemented the ability to edit existing tables within events, allowing for more flexibility in event management
	  ○ Added attendance marking functionality to track which users successfully attend events
	  ▪ We can link this to a QR system as well to make it automated from frontend
	  ○ Implemented public/private event visibility controls and event archiving features for better organization and access control
	  ○ Developed and verified login authentication logic to ensure users can securely access the system
	  ○ Added scheduled date and time fields to events to improve organization and clarity for users
	• Optimized Supabase database tables by refining schema design and making minor structural improvements
	  ○ Reduced unnecessary complexity in the events table by removing unused or redundant columns
	  ▪ Simplified key event attributes such as table size,

	capacity, and number of queues to improve maintainability and performance
	Planned tasks for next week:
	• Further optimize backend function logic to improve performance and reduce redundancy
	• Identify and fix edge cases that may impact user experience or system reliability
	  ○ Handle duplicate party creation scenarios (e.g., when a user tries to create a party for an event they've already joined)
	  ▪ Implement user prompts to either overwrite an existing party or cancel the action
	  ▪ Explore adding functionality to edit existing party reservations for greater flexibility
	• Reevaluate and potentially enhance login verification logic to improve security and scalability beyond the current basic implementation
	
	Any issues or challenges:
	• Refactoring some of the Event Creation/Queue Creation functions was a hassle since I didnt want to need to make seperate functions for SingleQ and MultiQ, and also wanted the same parameters to be sent to the same function, and it should just know which one you want

Total Time Spent: ~58 hours

Summary:

We successfully transitioned the project from a prototype into a production-ready system by overhauling the backend and infrastructure. We migrated from in-memory storage to Supabase PostgreSQL, enabling persistent, real-time data handling across authentication, events, and waitlists. We also cleaned and optimized the database schema, resolved duplicate tables, and reduced unnecessary complexity. At the same time, we rebuilt authentication using stateless HMAC-signed tokens, replacing localStorage-based methods with secure API-driven validation, resulting in a fully functional and reliable signup/login system.

On the frontend, we stabilized and debugged the application while fully integrating it with the live backend. We improved UI/UX across the platform with cleaner event cards, consistent layouts, and more intuitive navigation. We added features such as public/private event toggles, event-type badges, improved date/time displays, and simplified page flows. We also expanded and tested the event and queue system to support capacity-based, table-based, and attendance-

based events, with real-time synchronization, automated multi-queue generation, and full staff controls for managing seating, editing events, and tracking attendance.

In addition, we developed and integrated an advanced AI-powered wait time prediction system using Python/FastAPI. This system includes dynamic service time adjustments, adaptive no-show analytics, user reliability scoring, and frequent recalculations to maintain accuracy. We deployed the backend on Render and the frontend on Vercel, configured environment variables across both platforms, and maintained a backup repository for safe development. Through extensive testing from both user and staff perspectives, we confirmed that all core features—from event creation to queue management—are functioning correctly in the deployed environment.